

Documentazione Effhector

Effhector è un processore audio VST/AU sviluppato con il framework JUCE, che implementa tre effetti audio in tempo reale:

- Delay/eco digitale con feedback e interpolazione quadratica
- Chorus con LFO modulabile
- Riverbero basato sull'algoritmo di JUCE Reverb

Il plugin utilizza **Foleys Magic GUI** per l'interfaccia utente e supporta il salvataggio/ripristino dello stato.

Architettura del Codice

Classi Principali

EffhectorAudioProcessor

- Classe principale che eredita da `foleys::MagicProcessor`
- Gestisce tutto il processing audio e i parametri

File Dipendenze

- `PluginProcessor.h` - Dichiarazione della classe
- `PluginEditor.h` - Editor basato su Foleys Magic
- `GUI_SaveData.xml` - File XML per la configurazione GUI

Componenti Principali

1. Parametri Audio (AudioProcessorValueTreeState)

Delay

Parametri	Range	Default	Descrizione
DELAYMS	0-1000ms	0ms	tempo di delay
FEEDBACK	0-0.95	0.3	Feedback del delay

DRY	0.1-1.0	1.0	Livello segnale dry
WET	0.1-1.0	0.5	Livello segnale wet
ENABLE_DELAY	bool	false	attiva/disattiva il delay

Chorus

CHORUS_RATE	0.01-5 Hz	0.3 Hz	Frequenza LFO
CHORUS_DEPTH	0.1-10 ms	4.0 ms	Profondità modulazione
CHORUS_DELAY	1-20 ms	8.0 ms	Delay base chorus
CHORUS_MIX	0.0-1.0	0.3	Mix segnale chorus
ENABLE_CHORUS	bool	false	Attiva/disattiva chorus

Reverb

REVERB_MIX	0.0-1.0	0.3	Mix riverbero
REVERB_ROOM	0.0-1.0	0.3	Dimensione Stanza
REVERB_DAMPING	0.0-1.0	0.3	Smorzamento
REVERB_WIDTH	0.0-1.0	0.3	Larghezza stereo
ENABLE_REVERB	bool	false	Attiva/disattiva riverbero

2. Buffer e Strutture Dati

```
cpp
// Buffer principale delay (10 secondi)
juce::AudioBuffer<float> delayBuffer;

// Buffer chorus (50ms)
juce::AudioBuffer<float> chorusBuffer;
```

```
// Buffer ausiliario per riverbero
juce::AudioBuffer<float> revBuffer;

// Oggetto riverbero JUCE
juce::Reverb reverb;
juce::Reverb::Parameters revParams;

// Posizioni di scrittura
int writePos;      // Delay buffer
int chorusWritePos; // Chorus buffer
```

3. Variabili di Stato

- sampleRate: Frequenza di campionamento corrente
- currentDelaySamples: Delay corrente (con smoothing)
- targetDelaySamples: Delay target
- chorusPhase: Fase corrente LFO chorus
- isCrossfading: Flag crossfade cambio delay
- usingA: Flag buffer attivo (A/B per crossfade)

Algoritmi di Processing

Interpolazione

```
// Interpolazione lineare per chorus
static float readWithLerp(...)
```

```
// Interpolazione quadratica per delay (migliore qualità)
static float readWithQlerp(...)
```

Crossfade Delay

Il plugin implementa un sistema di crossfade per cambiamenti smooth del tempo di delay:

- Usa due buffer virtuali (A e B) con delay differenti
- Crossfade progressivo durante il cambiamento
- Previene click e artefatti

Processing Pipeline

1. **Lettura parametri e aggiornamento stati**
2. **Processamento campione-per-campione:**
 - Smoothing parametro delay
 - Calcolo LFO chorus
 - Lettura buffer con interpolazione
 - Scrittura buffer con feedback
 - Mix dry/wet
3. **Applicazione riverbero (se abilitato)**
4. **Aggiornamento posizioni buffer**

Flusso di Esecuzione

prepareToPlay(double sr, int samplesPerBlock)

- Inizializza buffer con dimensione basata sul sample rate
- Configura parametri default riverbero
- Resetta stati e posizioni

processBlock(...)

Per ogni campione nel buffer:

- ├─ Aggiorna smoothing delay
- ├─ Calcola LFO chorus
- ├─ Per ogni canale:
 - ├─ Leggi input
 - ├─ Processa delay (con crossfade se necessario)
 - ├─ Processa chorus
 - └─ Calcola output mix
- ├─ Incrementa posizioni buffer
- └─ Aggiorna crossfade

Se riverbero abilitato:

- ├─ Aggiorna parametri reverb
- ├─ Copia buffer in revBuffer
- ├─ Processa con reverb JUCE
- └─ Sostituisci output

releaseResources()

- Pulisce e resetta tutti i buffer
- Resetta stato riverbero
- Ripristina posizioni a zero

Gestione Stato e Preset

Salvataggio Stato

void getStateInformation(juce::MemoryBlock& destData)

- Serializza parametri AudioProcessorValueTreeState in XML
- Salva stato GUI Foleys
- Inserisce tutto nel MemoryBlock

Ripristino Stato

void setStateInformation(const void* data, int sizeInBytes)

- Deserializza XML da MemoryBlock
- Ripristina parametri AudioProcessorValueTreeState
- Ripristina stato GUI Foleys

Configurazione GUI (Foleys Magic)

Caricamento GUI

// Cerca GUI_SaveData.xml nella stessa cartella del codice

auto guiFile = sourceFolder.getChildFile("GUI_SaveData.xml");

- Il file XML deve essere presente nella cartella del progetto
- Se non trovato, stampa messaggio di debug

Disabilitazione Salvataggio Automatico

magicState.setApplicationSettingsFile(juce::File());

- Il plugin non salva automaticamente un nuovo file XML
- Lo stato viene gestito tramite getStateInformation/setStateInformation

Considerazioni Tecniche

Performance

- Buffer delay: 10 secondi massimi ($sr \times 10$)
- Buffer chorus: 50ms massimi ($sr \times 0.05$)
- Interpolazione quadratica: Migliore qualità ma più costosa
- Crossfade: Gestito su 50ms ($crossfadeTime = 0.05f$)

Limitazioni

- Supporta solo mono/stereo (verifica `isBusesLayoutSupported`)
- Tail length = 0.0 (nessun tail quando disabilitato)
- 1 programma/preset disponibile

Gestione Artefatti

- Smoothing coefficient per cambiamenti delay: `smoothingCoeff = 0.0005`
- Clipping su feedback: `juce::jlimit(-1.0f, 1.0f, toWrite)`
- Reset completo in `releaseResources()` per evitare artefatti

Integrazione Host

Formati Supportati

- VST2/VST3
- Audio Unit (AU)
- AAX (se configurato)
- Standalone (tramite JUCE)

Configurazioni Canali

- Mono → Mono
- Stereo → Stereo
- No supporto per configurazioni multicanale complesse